LAPORAN PRAKTIKUM

TEKNOLOGI IoT 2

"Protokol Komunikasi Telemetri Industri (MQTT)"

DISUSUN OLEH

Nama : Naurah Miftah Frizkiyah

Kelas : TRI 3B

NPM: 25782049

DOSEN :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.,

Andi Saptono, A.Md

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

6.6 Latihan

Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):

Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:

i. Efisiensi penggunaan kapasitas *bandwidth* pada paket data seluler.
ii. Beban komputasi mikrokontroler (*CPU overhead*) saat menangani pertukaran data dua arah.
iii. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (*1-to-N distribution*).

Jawab:
Protokol MQTT jauh lebih unggul daripada HTTP dalam hal efisiensi penggunaan kapasitas bandwidth pada paket data seluler karena MQTT menggunakan struktur biner yang sangat ringkas dengan ukuran fixed header minimum hanya sebesar dua byte saja. Sebaliknya, protokol HTTP berbasis teks memiliki overhead header yang sangat besar, berkisar antara 200 hingga 800 byte per permintaan, karena harus membawa informasi metode, status, dan berbagai atribut tambahan secara berulang. Dari sisi beban komputasi mikrokontroler (CPU overhead), HTTP menggunakan model Request-Response yang bersifat stateless sehingga mikrokontroler dipaksa terus-menerus membuka dan menutup koneksi TCP baru untuk setiap siklus pengiriman data, yang tentu sangat menguras sumber daya SRAM dan CPU. Sementara itu, MQTT mampu mempertahankan satu koneksi TCP yang persisten melalui fungsi pemeliharaan Keep-Alive Ping yang ringan, serta menerapkan komunikasi berbasis Publish-Subscribe dan fungsi callback asinkron sehingga mikrokontroler hanya memproses data saat ada paket masuk tanpa mengganggu alur utama program. Selain itu, dalam hal kemampuan pengiriman data secara simultan ke banyak perangkat penerima (1-to-N distribution), HTTP menggunakan komunikasi point-to-point langsung yang mengharuskan pengirim melakukan banyak permintaan terpisah ke setiap aplikasi pemantau, sedangkan MQTT mengandalkan MQTT Broker sebagai perute pesan terpusat di mana satu kali pengiriman dari perangkat dapat didistribusikan secara otomatis dan simultan ke banyak subscriber tanpa membebani mikrokontroler.

Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):

Sebuah pabrik manufaktur memiliki 3 lini produksi (*line1*, *line2*, *line3*), di mana setiap lini memiliki 4 mesin (*m1*, *m2*, *m3*, *m4*), dan setiap mesin melaporkan metrik suhu, getaran, dan status. Struktur topiknya adalah pabrik/<line>/<mesin>/<sensor>. Tentukan ekspresi topik subskripsi (*subscription topic*) yang tepat menggunakan karakter *wildcard* (+ atau #) untuk:

i. Memantau parameter suhu dari seluruh mesin di semua lini produksi.
Jawab:
Ekspresi Topik: pabrik/+/+/suhu
Menggunakan karakter *single-level wildcard* (+) untuk menggantikan posisi lini (line1, line2, line3) dan posisi mesin (m1, m2, m3, m4), serta menetapkan tingkat akhir secara spesifik pada parameter suhu.

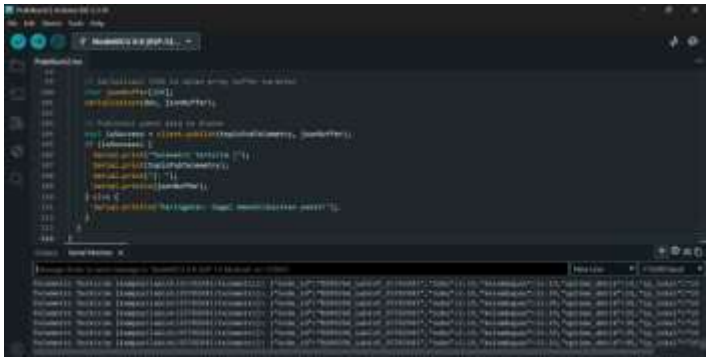
ii. Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2.
Jawab:
Ekspresi Topik: pabrik/line2/m1/#
Mengunci jalur secara spesifik pada line2 dan mesin m1, lalu menggunakan *multi-level wildcard* (#) di bagian akhir untuk menangkap seluruh sub-parameter sensor di bawahnya (suhu, getaran, status).

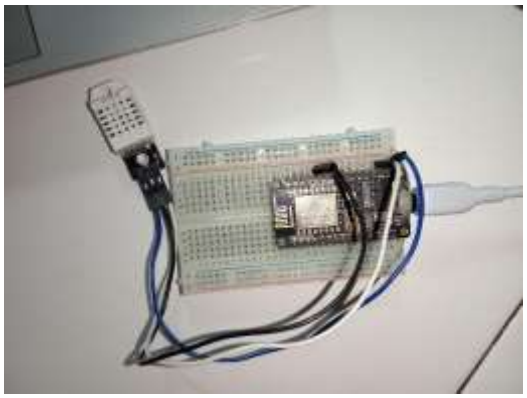
iii. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut.
Jawab:
Ekspresi Topik: `pabrik/#`
Menempatkan *multi-level wildcard* (`#`) tepat setelah akar direktori `pabrik/` untuk menyadap seluruh pesan, hierarki turunan, lini, mesin, dan jenis sensor apa pun secara menyeluruh.

Latihan 3 (Implementasi Serialisasi Data Dinamis):

Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:

i. "uptime_detik": Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus millis() / 1000).
ii. "ip_lokal": Alamat IP lokal mikrokontroler dalam format string (WiFi.localIP().toString()).
Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!





dilakukan penambahan dua parameter dinamis ke dalam objek dokumen JSON telemetri untuk memperkaya informasi status perangkat mikrokontroler yang dikirimkan ke broker MQTT.

- **uptime_detik (millis() / 1000):** Parameter ini berfungsi untuk merekam lama waktu aktif sistem (*uptime*) sejak mikrokontroler pertama kali dinyalakan atau di-*reset*. Fungsi bawaan millis() mengembalikan nilai milidetik yang terus bertambah dari waktu ke waktu, sehingga pembagian dengan nilai 1000 diperlukan untuk mengonversinya ke dalam satuan detik. Informasi ini sangat penting dalam pemantauan industri untuk mengetahui stabilitas operasional perangkat dan mendeteksi apakah terjadi *restart* atau kegagalan daya secara tiba-tiba.
- **ip_lokal (WiFi.localIP().toString()):** Parameter ini berfungsi untuk menyertakan alamat IP lokal yang sedang disematkan oleh jaringan Wi-Fi ke mikrokontroler ESP8266 dalam bentuk teks string. Hal ini berguna bagi operator untuk melacak identitas jaringan atau subnet tempat perangkat terhubung secara langsung melalui paket data telemetri yang masuk.

Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):

Pada Praktikum 3, di dalam fungsi loop() terdapat pemanggilan rutin client.loop().

i. Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah *callback* jika seorang pengembang menyisipkan instruksi delay(6000) di dalam fungsi loop()?

Jika pengembang menyisipkan instruksi delay(6000) di dalam fungsi loop(), mikrokontroler akan berada dalam kondisi *blocking* (berhenti total) selama 6 detik penuh pada setiap siklusnya. Akibat teknisnya:

- **Kegagalan *Keep-Alive Ping*:** Fungsi client.loop() tidak dapat dipanggil secara rutin untuk menjaga sesi TCP dengan broker MQTT, sehingga broker akan menganggap perangkat terputus karena *timeout*.
- **Penurunan Responsivitas Ekstrem:** Perintah kendali yang dikirimkan oleh operator (seperti mengubah status LED) tidak akan bisa diterima atau diproses secara seketika (*asinkron*), melainkan harus menunggu fungsi *delay* selesai dan koneksi broker pulih kembali.

ii. Jelaskan bagaimana fungsi reconnect() bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat!

Fungsi reconnect() bekerja secara berulang (*looping* bersyarat) menggunakan pengecekan client.connected(). Ketika sambungan terputus:

Sistem mendeteksi kegagalan status dan mencetak kode galat (*return code/rc*) ke Serial Monitor untuk keperluan diagnostik. Fungsi ini melakukan upaya jabat tangan ulang (*handshake*) ke broker menggunakan metode client.connect(clientId). Seketika koneksi berhasil dipulihkan, fungsi ini langsung mengeksekusi ulang perintah pendaftaran subskripsi topik (seperti client.subscribe()) agar perangkat kembali siap mendengarkan instruksi kendali tanpa perlu melakukan *restart* perangkat secara manual.